

# 详细设计说明书

## DBMS 控制台项目

### 目录

|       |                                    |    |
|-------|------------------------------------|----|
| 1     | 详细设计                               | 5  |
| 1.1   | 1. 文档定位                            | 5  |
| 1.2   | 2. 详细设计目标                          | 6  |
| 1.2.1 | DD-001 实现可追踪                       | 6  |
| 1.2.2 | DD-002 实现可分阶段                      | 6  |
| 1.2.3 | DD-003 实现可维护                       | 6  |
| 1.2.4 | DD-004 实现安全默认                      | 6  |
| 1.2.5 | DD-005 静态设计与动态设计显式分层               | 6  |
| 1.3   | 3. 后端详细设计                          | 7  |
| 1.3.1 | 3.1 推荐目录结构                         | 7  |
| 1.3.2 | 3.2 请求处理链路                         | 9  |
| 1.3.3 | 3.3 traceMiddleware                | 9  |
| 1.3.4 | 3.4 authMiddleware                 | 10 |
| 1.3.5 | 3.5 errorMiddleware                | 10 |
| 1.4   | 4. 配置与限制详细设计                       | 11 |
| 1.4.1 | 4.1 env.ts                         | 11 |
| 1.4.2 | 4.2 limits.ts                      | 11 |
| 1.4.3 | 4.3 limitService                   | 12 |
| 1.5   | 5. 认证与会话详细设计                       | 12 |
| 1.5.1 | 5.1 authService                    | 12 |
| 1.5.2 | 5.2 tokenService                   | 12 |
| 1.5.3 | 5.3 登录流程                           | 13 |
| 1.5.4 | 5.4 Refresh Token Rotation         | 13 |
| 1.6   | 6. 用户工作台详细设计                       | 13 |
| 1.6.1 | 6.1 workbenchService               | 13 |
| 1.6.2 | 6.2 preferenceService              | 14 |
| 1.6.3 | 6.3 assetService                   | 14 |
| 1.7   | 7. 数据库管理详细设计                       | 15 |
| 1.7.1 | 7.1 databaseService.createDatabase | 15 |
| 1.7.2 | 7.2 schemaName 生成                  | 16 |
| 1.7.3 | 7.3 databaseResolver               | 16 |

|        |                                    |    |
|--------|------------------------------------|----|
| 1.8    | 8. 元数据与表结构详细设计                     | 16 |
| 1.8.1  | 8.1 metadataService                | 16 |
| 1.8.2  | 8.2 identifier 工具                  | 17 |
| 1.8.3  | 8.3 schemaBuilder                  | 17 |
| 1.9    | 9. 表数据详细设计                         | 17 |
| 1.9.1  | 9.1 rowService.queryRows           | 17 |
| 1.9.2  | 9.2 行内编辑草稿机制                       | 18 |
| 1.9.3  | 9.3 rowService.updateRows          | 19 |
| 1.9.4  | 9.4 无主键表策略                         | 19 |
| 1.9.5  | 9.5 V5 当前实现范围                      | 20 |
| 1.10   | 10. 结构化查询详细设计                      | 20 |
| 1.10.1 | 10.1 查询 AST                        | 20 |
| 1.10.2 | 10.2 queryValidator                | 21 |
| 1.10.3 | 10.3 查询作用域                         | 22 |
| 1.10.4 | 10.4 queryBuilder                  | 22 |
| 1.10.5 | 10.5 子查询支持范围                       | 22 |
| 1.11   | 11. 事务详细设计                         | 23 |
| 1.11.1 | 11.1 transactionService            | 23 |
| 1.11.2 | 11.2 开启事务                          | 23 |
| 1.11.3 | 11.3 事务内操作                         | 24 |
| 1.11.4 | 11.4 超时回滚                          | 24 |
| 1.12   | 12. 统计详细设计                         | 24 |
| 1.12.1 | 12.1 statsService.userOverview     | 24 |
| 1.12.2 | 12.2 statsService.databaseOverview | 24 |
| 1.12.3 | 12.3 statsService.tableOverview    | 25 |
| 1.13   | 13. 审计详细设计                         | 25 |
| 1.13.1 | 13.1 auditService                  | 25 |
| 1.13.2 | 13.2 审计内容脱敏                        | 26 |
| 1.14   | 14. 前端详细设计                         | 26 |
| 1.14.1 | 14.1 推荐目录结构                        | 26 |
| 1.14.2 | 14.2 authStore                     | 28 |
| 1.14.3 | 14.3 request.ts                    | 28 |
| 1.14.4 | 14.4 WorkbenchLayout               | 28 |
| 1.14.5 | 14.5 DbTree                        | 29 |
| 1.14.6 | 14.6 DataGrid                      | 29 |
| 1.14.7 | 14.7 QueryBuilderPanel             | 30 |
| 1.14.8 | 14.8 DashboardWidget               | 30 |
| 1.15   | 15. 前后端交互流程                        | 31 |
| 1.15.1 | 15.1 登录后进入工作台                      | 31 |
| 1.15.2 | 15.2 创建数据库                         | 31 |
| 1.15.3 | 15.3 行内编辑提交                        | 31 |
| 1.15.4 | 15.4 结构化查询                         | 32 |
| 1.16   | 16. 开发落地顺序                         | 32 |

|          |                                          |           |
|----------|------------------------------------------|-----------|
| 1.16.1   | Phase 1: 认证与工作台地基 . . . . .              | 32        |
| 1.16.2   | Phase 2: 用户偏好和资源 . . . . .               | 32        |
| 1.16.3   | Phase 3: 数据库管理 . . . . .                 | 32        |
| 1.16.4   | Phase 4: 表结构管理 . . . . .                 | 33        |
| 1.16.5   | Phase 5: 表数据 CRUD . . . . .              | 33        |
| 1.16.6   | Phase 6: GUI 查询构造器 . . . . .             | 33        |
| 1.16.7   | Phase 7: 统计和 DIY 工作台 . . . . .           | 34        |
| 1.17     | 17. 实现检查清单 . . . . .                     | 35        |
| 1.18     | 18. 非功能性设计 . . . . .                     | 35        |
| 1.18.1   | 18.1 性能设计 . . . . .                      | 35        |
| 1.18.2   | 18.2 可用性设计 . . . . .                     | 35        |
| 1.18.3   | 18.3 可维护性设计 . . . . .                    | 36        |
| 1.18.4   | 18.4 可观测性设计 . . . . .                    | 36        |
| 1.18.5   | 18.5 可扩展性设计 . . . . .                    | 36        |
| 1.19     | 19. 防御性设计 . . . . .                      | 36        |
| 1.19.1   | 19.1 输入防御 . . . . .                      | 36        |
| 1.19.2   | 19.2 SQL 防御 . . . . .                    | 36        |
| 1.19.3   | 19.3 权限防御 . . . . .                      | 36        |
| 1.19.4   | 19.4 状态防御 . . . . .                      | 37        |
| 1.19.5   | 19.5 容量防御 . . . . .                      | 37        |
| 1.20     | 20. 详细设计追踪 . . . . .                     | 37        |
| <b>2</b> | <b>数据库设计附录</b>                           | <b>37</b> |
| <b>3</b> | <b>数据库设计</b>                             | <b>37</b> |
| 3.1      | 1. 设计原则 . . . . .                        | 37        |
| 3.1.1    | DATA-001 系统库和用户物理库分离 . . . . .           | 37        |
| 3.1.2    | DATA-002 第三范式 . . . . .                  | 38        |
| 3.1.3    | DATA-003 用户表结构自由 . . . . .               | 38        |
| 3.1.4    | DATA-004 真实物理库名后端生成 . . . . .            | 38        |
| 3.2      | 2. 系统库表总览 . . . . .                      | 38        |
| 3.3      | 3. 命名规则 . . . . .                        | 38        |
| 3.4      | 4. users 用户认证表 . . . . .                 | 39        |
| 3.5      | 5. user_profiles 用户资料表 . . . . .         | 39        |
| 3.6      | 6. user_preferences 用户偏好表 . . . . .      | 39        |
| 3.7      | 7. user_assets 用户资源元信息表 . . . . .        | 40        |
| 3.8      | 8. refresh_tokens 会话刷新表 . . . . .        | 41        |
| 3.9      | 9. user_databases 用户数据库归属表 . . . . .     | 41        |
| 3.10     | 10. transaction_sessions 事务会话表 . . . . . | 42        |
| 3.11     | 11. audit_logs 操作日志表 . . . . .           | 43        |
| 3.12     | 12. stat_snapshots 统计快照表 . . . . .       | 45        |
| 3.13     | 13. MySQL 元数据来源 . . . . .                | 45        |
| 3.14     | 14. 支持的数据类型 . . . . .                    | 47        |

|        |                                     |    |
|--------|-------------------------------------|----|
| 3.15   | 15. 索引设计                            | 48 |
| 3.15.1 | DATA-IDX-001 索引设计原则                 | 48 |
| 3.15.2 | DATA-IDX-002 系统库索引清单                | 48 |
| 3.15.3 | DATA-IDX-003 用户物理库索引策略              | 49 |
| 3.15.4 | DATA-IDX-004 索引变更安全                 | 49 |
| 3.15.5 | DATA-IDX-005 索引统计来源                 | 49 |
| 3.16   | 16. 容量限制                            | 50 |
| 3.16.1 | DATA-LIMIT-001 限制来源                 | 50 |
| 3.16.2 | DATA-LIMIT-002 默认容量限制               | 50 |
| 3.16.3 | DATA-LIMIT-003 容量统计原则               | 51 |
| 4      | 安全设计附录                              | 51 |
| 5      | 安全设计                                | 51 |
| 5.1    | 1. 安全目标                             | 51 |
| 5.2    | 2. 双 Token 安全                       | 51 |
| 5.2.1  | SEC-AUTH-001 Access Token           | 51 |
| 5.2.2  | SEC-AUTH-002 Refresh Token          | 52 |
| 5.2.3  | SEC-AUTH-003 Refresh Token Rotation | 52 |
| 5.2.4  | SEC-AUTH-004 退出登录                   | 53 |
| 5.3    | 3. 授权安全                             | 53 |
| 5.3.1  | SEC-AUTHZ-001 用户上下文                 | 53 |
| 5.3.2  | SEC-AUTHZ-002 数据库归属校验               | 53 |
| 5.3.3  | SEC-AUTHZ-003 对象归属继承                | 53 |
| 5.3.4  | SEC-AUTHZ-004 事务归属校验                | 53 |
| 5.4    | 4. SQL 安全                           | 53 |
| 5.4.1  | SEC-SQL-001 值参数化                    | 53 |
| 5.4.2  | SEC-SQL-002 标识符校验                   | 54 |
| 5.4.3  | SEC-SQL-003 标识符来源校验                 | 54 |
| 5.4.4  | SEC-SQL-004 统一转义                    | 54 |
| 5.4.5  | SEC-SQL-005 GUI 查询限制                | 54 |
| 5.4.6  | SEC-SQL-006 写操作限制                   | 55 |
| 5.5    | 5. 高风险操作确认                          | 55 |
| 5.5.1  | SEC-CONFIRM-001 强制确认                | 55 |
| 5.5.2  | SEC-CONFIRM-002 偏好确认                | 55 |
| 5.5.3  | SEC-CONFIRM-003 确认失败                | 56 |
| 5.6    | 6. 事务安全                             | 56 |
| 5.6.1  | SEC-TX-001 事务连接                     | 56 |
| 5.6.2  | SEC-TX-002 事务超时                     | 56 |
| 5.6.3  | SEC-TX-003 隔离级别                     | 56 |
| 5.6.4  | SEC-TX-004 审计                       | 56 |
| 5.7    | 7. 精确错误安全                           | 56 |
| 5.7.1  | SEC-ERROR-001 错误归类                  | 56 |

|        |                          |    |
|--------|--------------------------|----|
| 5.7.2  | SEC-ERROR-002 MySQL 错误返回 | 57 |
| 5.7.3  | SEC-ERROR-003 批量操作错误     | 57 |
| 5.8    | 8. 文件资源安全                | 57 |
| 5.8.1  | SEC-FILE-001 资源存储        | 57 |
| 5.8.2  | SEC-FILE-002 文件校验        | 57 |
| 5.8.3  | SEC-FILE-003 文件名         | 58 |
| 5.8.4  | SEC-FILE-004 资源归属        | 58 |
| 5.9    | 9. 容量与频率限制               | 58 |
| 5.9.1  | SEC-LIMIT-001 容量限制       | 58 |
| 5.9.2  | SEC-LIMIT-002 频率限制       | 59 |
| 5.9.3  | SEC-LIMIT-003 登录失败冷却     | 59 |
| 5.10   | 10. 审计与追踪                | 59 |
| 5.10.1 | SEC-AUDIT-001 traceId    | 59 |
| 5.10.2 | SEC-AUDIT-002 审计范围       | 60 |
| 5.10.3 | SEC-AUDIT-003 审计内容       | 60 |
| 5.11   | 11. 环境变量                 | 61 |
| 5.12   | 12. 前端安全                 | 61 |
| 5.13   | 13. 防御性设计汇总              | 61 |
| 5.13.1 | SEC-DEF-001 默认拒绝         | 61 |
| 5.13.2 | SEC-DEF-002 失败安全         | 61 |
| 5.13.3 | SEC-DEF-003 最小暴露         | 62 |
| 5.13.4 | SEC-DEF-004 可追踪          | 62 |

## 1 详细设计

### 1.1 1. 文档定位

本文档承接以下文档：

- 需求规格说明书.md：说明系统要实现什么。
- 概要设计.md：说明系统总体怎么拆分。
- 数据库设计.md：说明系统数据如何持久化。
- 接口规范.md：说明前后端如何通信。
- 安全设计.md：说明认证、授权、SQL、文件、容量限制等安全规则。

本文档重点说明“如何落地实现”，包括模块职责、关键数据结构、核心流程、伪代码、状态设计和实现顺序。

## 1.2 2. 详细设计目标

### 1.2.1 DD-001 实现可追踪

每个核心模块都应能追踪到需求编号、接口编号、安全编号或数据设计编号。

### 1.2.2 DD-002 实现可分阶段

设计必须支持从最小闭环逐步扩展：

认证与工作台

- > 数据库管理
- > 表结构管理
- > 表数据 CRUD
- > 查询构造器
- > 事务
- > 统计和 DIY 工作台

### 1.2.3 DD-003 实现可维护

后端 controller 不直接拼 SQL，不直接写复杂业务；前端页面不直接堆全部逻辑，复杂逻辑下沉到 composable、store 和组件。

### 1.2.4 DD-004 实现安全默认

所有数据库、表、字段、视图、索引、约束、资源都必须基于当前用户上下文做归属校验。

### 1.2.5 DD-005 静态设计与动态设计显式分层

详细设计同时包含静态设计和动态设计。

静态设计描述系统由哪些结构组成，包括：

| 静态设计内容                                    | 所在位置        |
|-------------------------------------------|-------------|
| 后端目录、controller、service、repository、SQL 工具 | 本文档“后端详细设计” |
| 前端目录、store、composable、layout、component    | 本文档“前端详细设计” |
| 系统库表、字段、约束、索引                             | 数据库设计.md    |
| 接口路径、请求、响应、错误码                            | 接口规范.md     |

动态设计描述系统运行时如何流转，包括：

| 动态设计内容                 | 所在位置           |
|------------------------|----------------|
| 请求处理链路                 | 本文档“请求处理链路”    |
| 登录、刷新 Token、退出登录       | 本文档“认证与会话详细设计” |
| 创建数据库和失败补偿             | 本文档“数据库管理详细设计” |
| 行内编辑草稿、提交、取消           | 本文档“表数据详细设计”   |
| 查询 AST 校验、作用域解析、SQL 生成 | 本文档“结构化查询详细设计” |
| 事务开启、提交、回滚、超时回滚        | 本文档“事务详细设计”    |
| 前后端交互流程                | 本文档“前后端交互流程”   |

### 1.3 3. 后端详细设计

#### 1.3.1 3.1 推荐目录结构

```

backend/src/
  app.ts
  server.ts

config/
  env.ts
  db.ts
  limits.ts

middlewares/
  authMiddleware.ts
  errorMiddleware.ts
  traceMiddleware.ts
  rateLimitMiddleware.ts

routes/
  authRoutes.ts
  workbenchRoutes.ts
  databaseRoutes.ts
  tableRoutes.ts
  rowRoutes.ts
  queryRoutes.ts
  transactionRoutes.ts
  statsRoutes.ts
  auditRoutes.ts
  assetRoutes.ts

controllers/

```

```
    authController.ts
    workbenchController.ts
    databaseController.ts
    tableController.ts
    rowController.ts
    queryController.ts
    transactionController.ts
    statsController.ts
    auditController.ts
    assetController.ts

services/
    authService.ts
    tokenService.ts
    userService.ts
    preferenceService.ts
    assetService.ts
    databaseService.ts
    metadataService.ts
    tableService.ts
    rowService.ts
    queryService.ts
    transactionService.ts
    statsService.ts
    auditService.ts
    limitService.ts

repositories/
    userRepository.ts
    refreshTokenRepository.ts
    preferenceRepository.ts
    assetRepository.ts
    userDatabaseRepository.ts
    auditRepository.ts
    transactionRepository.ts

sql/
    identifier.ts
    escape.ts
    schemaBuilder.ts
    rowBuilder.ts
    queryAst.ts
    queryBuilder.ts
```



```
conditionBuilder.ts

types/
  auth.ts
  request.ts
  response.ts
  database.ts
  table.ts
  row.ts
  query.ts
  error.ts

utils/
  response.ts
  errors.ts
  crypto.ts
  time.ts
```

### 1.3.2 3.2 请求处理链路

所有业务请求按固定顺序处理：

HTTP Request

```
-> traceMiddleware 生成 traceId
-> rateLimitMiddleware 做基础频率限制
-> express.json / multipart 解析
-> authMiddleware 解析当前用户
-> route
-> controller 读取参数
-> service 执行业务逻辑
-> repository / sql builder / mysql pool
-> auditService 记录审计
-> response.success 返回
-> errorMiddleware 统一错误响应
```

### 1.3.3 3.3 traceMiddleware

职责：

- 为每个请求生成 traceId。
- 若前端传入 X-Trace-Id，后端可以复用合法 traceId。
- 将 traceId 写入请求上下文。
- 响应中必须返回 traceId。

伪代码：

```
function traceMiddleware(req, res, next) {
  req.traceId = normalizeTraceId(req.headers['x-trace-id']) ?? generateTraceId();
  res.setHeader('X-Trace-Id', req.traceId);
  next();
}
```

### 1.3.4 3.4 authMiddleware

职责：

- 从 Authorization: Bearer <accessToken> 读取 Access Token。
- 校验签名、有效期和 payload。
- 查询用户状态是否可用。
- 将 currentUser 写入请求上下文。

请求上下文建议：

```
type CurrentUser = {
  userID: number;
  accountNo: string;
  status: 'active' | 'disabled';
};
```

认证失败返回：

HTTP 401

error.code = AUTH\_TOKEN\_MISSING / AUTH\_TOKEN\_EXPIRED / AUTH\_TOKEN\_INVALID

### 1.3.5 3.5 errorMiddleware

职责：

- 捕获 controller、service、repository 抛出的错误。
- 将系统错误转换为 接口规范 .md 的统一错误响应。
- MySQL 错误需要映射为明确错误码。
- 不返回密码、Token、真实物理库名、服务端文件绝对路径。

错误类型建议：

```
class AppError extends Error {
  httpStatus: number;
  type: string;
  code: string;
  details?: string;
  fieldErrors?: FieldError[];
```

```
mysql?: MysqlSafeError | null;  
}
```

## 1.4 4. 配置与限制详细设计

### 1.4.1 4.1 env.ts

env.ts 统一读取环境变量。关键环境变量缺失时服务必须启动失败。

必需项:

DB\_HOST  
DB\_USER  
DB\_PASSWORD  
DB\_NAME  
JWT\_ACCESS\_SECRET  
JWT\_REFRESH\_SECRET

可选项:

PORT  
CORS\_ORIGIN  
UPLOAD\_ROOT  
COOKIE\_SECURE

### 1.4.2 4.2 limits.ts

容量限制集中定义:

```
export const limits = {  
  maxDatabasesPerUser: 10,  
  maxTablesPerDatabase: 100,  
  maxViewsPerDatabase: 50,  
  maxColumnsPerTable: 80,  
  maxIndexesPerTable: 16,  
  maxUserStorageBytes: 1024 * 1024 * 1024,  
  maxDatabaseStorageBytes: 512 * 1024 * 1024,  
  maxTableStorageBytes: 256 * 1024 * 1024,  
  maxPageSize: 100,  
  maxQueryRows: 1000,  
  maxBatchInsertRows: 500,  
  maxBatchWriteAffectedRows: 200,  
  maxJoinCount: 8,  
  maxSubqueryDepth: 3,  
  maxQueryDurationMs: 10000,  
}
```

```
maxQueryAstBytes: 64 * 1024,  
maxAvatarBytes: 2 * 1024 * 1024,  
maxBackgroundBytes: 8 * 1024 * 1024,  
maxUserAssetBytes: 100 * 1024 * 1024,  
};
```

### 1.4.3 4.3 limitService

职责:

- 创建数据库前检查单用户数据库数量。
- 创建表前检查单库表数量。
- 新增字段前检查字段数量。
- 创建索引前检查索引数量。
- 执行查询前检查 AST 体积、JOIN 数、子查询深度、pageSize。
- 批量写入前检查请求行数。
- 批量修改/删除前检查影响行数。
- 上传资源前检查单文件大小和用户资源总量。

超限错误:

```
throw new AppError({  
  httpStatus: 413,  
  type: 'LIMIT_EXCEEDED',  
  code: 'LIMIT_EXCEEDED',  
  details: '单用户数据库数量已达到上限: 10',  
});
```

## 1.5 5. 认证与会话详细设计

### 1.5.1 5.1 authService

职责:

- 注册用户。
- 登录校验。
- 更新最近登录时间。
- 调用 tokenService 签发双 Token。
- 调用 auditService 记录认证日志。

### 1.5.2 5.2 tokenService

职责:

- 生成 Access Token。
- 生成 Refresh Token。
- 哈希 Refresh Token 后写入 `refresh_tokens`。
- 刷新 Token 时执行 Refresh Token Rotation。
- 退出登录时撤销 Refresh Token。

### 1.5.3 5.3 登录流程

POST /api/v1/auth/login

- > 校验 accountNo、password
- > 查询 users
- > `bcrypt.compare`
- > 生成 accessToken, 15 分钟有效
- > 生成 refreshToken, 7/30 天有效
- > refreshToken 哈希写入 `refresh_tokens`
- > refreshToken 设置 HttpOnly Cookie
- > 返回 accessToken 和用户展示信息

### 1.5.4 5.4 Refresh Token Rotation

刷新流程:

POST /api/v1/auth/refresh

- > 从 HttpOnly Cookie 读取 refreshToken
- > 哈希后查询 `refresh_tokens`
- > 校验未过期、未撤销
- > 撤销旧 refreshToken
- > 新增新 refreshToken
- > 返回新 accessToken
- > 设置新 refreshToken Cookie

异常处理:

- 旧 token 不存在: 返回 401。
- 旧 token 已撤销但再次使用: 可能存在 token 盗用, 撤销同一 `token_family` 下所有 token。
- 用户状态 disabled: 返回 403。

## 1.6 6. 用户工作台详细设计

### 1.6.1 6.1 workbenchService

工作台概览聚合以下数据:

- 当前用户资料。

- 用户偏好。
- 用户资源引用。
- 默认 Dashboard 布局。
- 最近访问数据库。
- 最近操作日志。
- 用户级统计摘要。
- 数据库入口列表和数据库级摘要卡片。

首页职责边界：

- 首页只展示用户层、数据库层统计和数据库入口。
- 表结构、表数据、查询构造器等数据库内部操作统一进入 `DatabaseWorkbench.vue` 后完成。
- `Home.vue` 不直接承载表设计表单，避免总览页变成具体操作页。

### 1.6.2 6.2 preferenceService

管理：

- 主题模式。
- 主题色相。
- 背景类型。
- 背景资源。
- 默认分页大小。
- 二次确认偏好。

偏好更新必须只允许当前用户更新自己的记录。

### 1.6.3 6.3 assetService

资源类型：

avatar

background

存储原则：

- 文件实体保存到 `UPLOAD_ROOT`。
- 系统库 `user_assets` 保存元信息。
- 文件名由服务端随机生成。
- 读取资源必须校验 `asset.user_id = currentUserId`。
- 允许 png、jpeg、webp；SVG 固定拒绝，避免脚本、外链和内容嗅探风险。

## 1.7 7. 数据库管理详细设计

### 1.7.1 7.1 databaseService.createDatabase

输入:

```
type CreateDatabaseInput = {  
  userID: number;  
  displayName: string;  
  charset?: string;  
  collation?: string;  
};
```

流程:

1. 校验 displayName 标识符。
2. 检查当前用户数据库数量。
3. 检查当前用户 displayName 是否重复。
4. 生成 schemaName。
5. 执行 CREATE DATABASE。
6. 写入 user\_databases。
7. 写入 audit\_logs。
8. 返回 databaseId 和 displayName。

当前落地版本补充:

- 创建时真实物理库名格式为 u{userId}\_{safeDisplayName}\_{randomSuffix}。
- 如果随机后缀发生极低概率碰撞, 最多重试 5 次。
- 服务启动会自动检查 user\_databases 表, 避免开发环境漏执行新版初始化脚本。

补偿策略:

如果 CREATE DATABASE 成功但 user\_databases 写入失败:

- > 尝试 DROP DATABASE schemaName
- > 记录失败审计
- > 返回系统错误

删除数据库采用二次确认:

1. 前端要求用户输入完整数据库显示名。
2. 请求体传入 confirmation.confirmed=true 和 confirmation.confirmText。
3. 后端先把状态改为 deleting。
4. 删除物理 schema 成功后把状态改为 deleted。
5. 如果物理 schema 删除失败, 状态回滚为 active。

### 1.7.2 7.2 schemaName 生成

规则:

`u{userID}_{displayName}_{suffix}`

其中:

- `displayName` 已经符合标识符规则。
- `suffix` 使用短随机字符串, 防止重名冲突。
- `schemaName` 不返回前端。

### 1.7.3 7.3 databaseResolver

所有涉及数据库的接口必须先解析:

```
async function resolveUserDatabase(userID: number, databaseId: number) {
  const db = await userDatabaseRepository.findActiveByIdAndUser(databaseId, userID);
  if (!db) throw forbiddenOrNotFound();
  return db;
}
```

默认返回 403 还是 404:

- 如果资源存在但不属于当前用户, 不应暴露存在性, 推荐统一返回 404 或统一业务提示。
- 管理类项目为了调试清晰, 可以返回 403, 但不能包含真实物理库名。

## 1.8 8. 元数据与表结构详细设计

### 1.8.1 8.1 metadataService

职责:

- 查询表、视图列表。
- 查询字段列表。
- 查询主键。
- 查询索引。
- 查询约束。
- 判断表/视图/字段是否存在。

数据来源:

```
information_schema.tables
information_schema.columns
information_schema.statistics
information_schema.table_constraints
```



```
information_schema.key_column_usage
information_schema.views
```

### 1.8.2 8.2 identifier 工具

禁止业务代码直接拼接用户输入的数据库名、表名、字段名。

```
function assertIdentifier(value: string, label: string): void;
function quoteIdentifier(value: string): string;
function quotePath(parts: string[]): string;
```

规则:

```
^[A-Za-z][A-Za-z0-9_]{0,63}$
```

### 1.8.3 8.3 schemaBuilder

职责:

- 根据结构化表定义生成 CREATE TABLE。
- 根据结构化变更生成 ALTER TABLE。
- 控制字段类型白名单。
- 控制约束、索引、外键策略白名单。

CREATE TABLE 生成流程:

1. 校验 tableName。
2. 校验 columns 长度。
3. 对每个 column 校验 name、type、length、precision、scale、defaultValue。
4. 校验 constraints 引用字段存在。
5. 校验 indexes 引用字段存在。
6. 校验最多存在一个主键；无主键表允许创建，但数据编辑阶段必须按无主键表策略降级。
7. 生成字段定义 SQL。
8. 生成约束 SQL。
9. 生成索引 SQL。
10. 组合 CREATE TABLE。

## 1.9 9. 表数据详细设计

### 1.9.1 9.1 rowService.queryRows

职责:

- 分页预览表数据。
- 查询字段结构。

- 校验筛选字段。
- 限制 limit 和 offset。
- 返回 columns、rows、total、hasMore 和有限值 facets。

流程：

1. resolveUserDatabase。
2. 校验 tableName。
3. metadataService 确认表存在。
4. metadataService 查询字段和主键。
5. 校验 filters 是否来自真实字段。
6. 构造 SELECT 和 WHERE。
7. 查询 total。
8. 查询分页数据。
9. 查询有限离散字段候选值。
10. 返回。

### 1.9.2 9.2 行内编辑草稿机制

行内编辑由前端负责草稿态，后端只接收明确提交。

前端状态：

```
type RowDraftState = {
  rowKey: string;
  original: Record<string, unknown>;
  draft: Record<string, unknown>;
  changedFields: Record<string, unknown>;
  status: 'clean' | 'dirty' | 'saving' | 'error';
  errorMessage?: string;
};
```

交互流程：

用户修改单元格

- > 前端更新 draft
- > 对比 original 生成 changedFields
- > 行状态变为 dirty
- > 显示“更新”和“取消”

用户点击更新

- > 调用 PATCH /rows
- > 请求体包含 primaryKey + set
- > 成功后 original = draft
- > 行状态变为 clean

用户点击取消

```
-> draft = original  
-> changedFields 清空  
-> 行状态变为 clean
```

### 1.9.3 9.3 rowService.updateRows

单行更新：

1. resolveUserDatabase。
2. 校验 tableName。
3. 读取表结构和主键。
4. 校验 primaryKey 完整。
5. 校验 set 字段存在且不是禁止修改字段。
6. 检查单表容量限制。
7. 构造 UPDATE ... WHERE pk <=> ?。
8. 执行更新。
9. 影响行数必须等于 1。
10. 回读最新行数据。
11. 写入审计日志。

批量更新：

1. 校验 where GUI 条件对象。
2. 先执行 SELECT COUNT(\*) 预估影响行数。
3. 超过阈值时要求 confirmation。
4. 构造 UPDATE。
5. 执行并审计。

### 1.9.4 9.4 无主键表策略

第一阶段建议：

- 允许浏览无主键表。
- 允许新增数据。
- 禁止行内编辑。
- 禁止单行删除。
- 基于主键列表的批量修改/删除必须限制影响行数并进行二次确认；基于条件的批量修改/删除必须通过 GUI 条件构造器。

原因：

没有主键就无法稳定定位“用户正在编辑的那一行”

### 1.9.5 9.5 V5 当前实现范围

V5 已落地接口：

- GET /api/v1/databases/:databaseId/tables/:tableName/preview: 分页预览、字段筛选、有限值 facets。
- POST /api/v1/databases/:databaseId/tables/:tableName/rows: 新增单行和批量新增。
- PATCH /api/v1/databases/:databaseId/tables/:tableName/rows: 基于主键更新单行和基于主键列表批量修改。
- DELETE /api/v1/databases/:databaseId/tables/:tableName/rows: 基于主键删除单行和基于主键列表批量删除。
- PATCH /api/v1/databases/:databaseId/tables/:tableName/schema: 在线结构管理。

V5 已落地前端交互：

- 工作台主区域展示表格预览。
- 表头筛选留空表示全选，按 Enter 或刷新按钮后请求。
- 支持表头排序和列显示控制。
- 新增行在表格顶部进入草稿态，保存后刷新预览。
- 批量新增通过 JSON 数组弹窗提交。
- 行内编辑只修改本地草稿，点击“更新”后才提交。
- 批量修改通过已选主键列表和字段值弹窗提交。
- 批量删除通过已选主键列表和确认弹窗提交。
- 无主键表只允许浏览和新增，编辑与删除按钮禁用。

已补齐能力：

- 后端支持批量新增、批量修改和批量删除，并使用事务保证批次原子性。
- 前端支持排序、列显示控制、批量新增、基于主键列表的批量修改和批量删除。
- 结构管理支持字段新增、字段修改、字段删除、索引新增/删除和字符串有限取值检查约束。

后续增强：

- 基于 GUI 条件构造器的条件批量修改/删除。
- 级联影响分析和数据写操作审计日志的完整落库。

## 1.10 10. 结构化查询详细设计

### 1.10.1 10.1 查询 AST

前端不提交原始 SQL，而是提交查询 AST。

核心结构：

```
type SelectQueryAst = {  
  from: QuerySource;  
  joins?: JoinNode[];
```

```

fields: FieldNode[];
filters?: ConditionNode[];
groups?: FieldRef[];
having?: ConditionNode[];
sorts?: SortNode[];
page?: number;
pageSize?: number;
};

type QuerySource =
  | { type: 'table'; table: string; alias?: string }
  | { type: 'view'; view: string; alias?: string }
  | { type: 'subquery'; query: SelectQueryAst; alias: string };

```

条件节点:

```

type ConditionNode = {
  field?: string;
  rightField?: string;
  operator: 'eq' | 'ne' | 'gt' | 'gte' | 'lt' | 'lte' |
    'like' | 'in' | 'notIn' | 'between' |
    'isNull' | 'isNotNull' | 'exists' | 'notExists';
  value?: unknown;
  values?: unknown[];
  subquery?: SelectQueryAst;
  logic?: 'AND' | 'OR';
};

```

### 1.10.2 10.2 queryValidator

职责:

- 校验 AST 体积。
- 校验主表存在。
- 校验 JOIN 来源存在。
- 校验子查询深度不超过 3。
- 校验总 JOIN 数不超过 8。
- 校验字段引用在当前作用域存在。
- 校验聚合函数白名单。
- 校验操作符白名单。
- 校验分页上限。

### 1.10.3 10.3 查询作用域

查询构造器需要维护作用域：

scope:

```
alias -> source
alias.field -> column metadata
```

当遇到子查询：

1. 创建子 scope。
2. 注册子查询 from 和 joins。
3. 校验子查询 fields 和 filters。
4. 子查询输出字段作为派生表字段暴露给父 scope。

### 1.10.4 10.4 queryBuilder

生成 SQL 流程：

```
buildSelect(ast, context)
-> validate ast
-> build FROM
-> build JOIN
-> build SELECT fields
-> build WHERE
-> build GROUP BY
-> build HAVING
-> build ORDER BY
-> build LIMIT OFFSET
-> 返回 { sql, params }
```

所有字段、表名、别名使用 `quoteIdentifier`。所有值放入 `params`。

V6 落地实现：

- 后端新增 `queryController.executeSelectQuery`，挂载在 `POST /api/v1/databases/:databaseId/query/s`
- 后端按登录用户解析 `databaseId`，只允许访问当前用户拥有的物理 schema。
- 查询 AST 经过体积、来源、字段、操作符、聚合函数、分页、JOIN 数量和子查询深度校验。
- `rightField` 用于 JOIN ON 或字段间比较，普通值仍全部进入参数绑定。
- 顶层查询默认分页并返回 `hasMore`，前端通过加载下一页继续追加结果。
- 前端 `DatabaseWorkbench.vue` 新增“查询构造”模式，普通模式覆盖主表字段、JOIN、筛选、分组、HAVING、排序、分页；高级 AST 面板覆盖完整子查询节点。

### 1.10.5 10.5 子查询支持范围

第一阶段支持：

- 派生表: FROM (SELECT ...) alias
- IN (SELECT ...)
- NOT IN (SELECT ...)
- EXISTS (SELECT ...)
- NOT EXISTS (SELECT ...)
- 标量子查询: field > (SELECT AVG(...))

第一阶段不支持:

- CTE: WITH ...
- 窗口函数。
- 递归查询。
- UNION。
- 用户输入原始 SQL 片段。

这些能力后续可以作为高级查询功能扩展。

## 1.11 11. 事务详细设计

### 1.11.1 11.1 transactionService

事务会话绑定后端 MySQL 连接。

核心状态:

```
type TransactionSession = {  
  id: string;  
  userID: number;  
  databaseId: number;  
  schemaName: string;  
  connectionId: number;  
  status: 'active' | 'committed' | 'rolled_back' | 'expired';  
  isolationLevel: string;  
  expiresAt: Date;  
};
```

### 1.11.2 11.2 开启事务

1. resolveUserDatabase。
2. 从连接池获取连接。
3. USE `schemaName`。
4. SET TRANSACTION ISOLATION LEVEL ...
5. BEGIN。
6. 写入 transaction\_sessions。
7. 返回 transactionId。

### 1.11.3 11.3 事务内操作

写操作如果携带 `transactionId`:

1. 校验事务存在。
2. 校验事务属于当前用户。
3. 校验事务属于当前 `databaseId`。
4. 校验事务未过期。
5. 使用事务绑定连接执行 SQL。

### 1.11.4 11.4 超时回滚

事务超时后:

1. 自动 `ROLLBACK`。
2. 释放连接。
3. `transaction_sessions` 状态改为 `expired`。
4. 写入审计日志。

## 1.12 12. 统计详细设计

### 1.12.1 12.1 statsService.userOverview

数据来源:

- `user_databases`。
- `information_schema.tables`。
- `audit_logs`。
- `user_assets`。

返回:

- 数据库数量。
- 表数量。
- 视图数量。
- 索引数量。
- 估算行数。
- 操作趋势。
- 最近操作。

### 1.12.2 12.2 statsService.databaseOverview

流程:



1. resolveUserDatabase。
2. 查询 `information_schema.tables`。
3. 查询 `information_schema.columns`。
4. 聚合表数量、视图数量、数据大小、索引大小。
5. 统计字段类型分布。
6. 查询 `audit_logs` 得到最近操作。

### 1.12.3 12.3 statsService.tableOverview

表级统计涉及用户数据表，必须限制字段和执行时间。

支持：

- 行数估算。
- 字段数量。
- 主键信息。
- 空值比例。
- 数值字段 min、max、avg。
- 分类字段 Top N。

统计查询必须使用字段白名单，不允许前端传入聚合 SQL。

## 1.13 13. 审计详细设计

### 1.13.1 13.1 auditService

所有关键操作都调用：

```
auditService.record({
  traceId,
  userID,
  databaseId,
  tableName,
  actionType,
  targetType,
  summary,
  detail,
  success,
  errorMessage,
  durationMs,
});
```

### 1.13.2 13.2 审计内容脱敏

禁止写入：

- 密码。
- 密码哈希。
- Access Token。
- Refresh Token。
- Refresh Token 哈希。
- 文件绝对路径。
- 真实物理库名。

允许写入：

- 用户可见数据库名。
- 表名。
- 字段名。
- 影响行数。
- 错误码。
- traceId。

## 1.14 14. 前端详细设计

### 1.14.1 14.1 推荐目录结构

```
frontend/src/  
  assets/  
    theme.css  
    glass.css  
    global.css  
  
  router/  
    index.ts  
  
  utils/  
    request.ts  
    errorMessage.ts  
  
  api/  
    authApi.ts  
    workbenchApi.ts  
    databaseApi.ts  
    tableApi.ts  
    rowApi.ts
```

```
    queryApi.ts
    statsApi.ts
    assetApi.ts

stores/
  authStore.ts
  preferenceStore.ts
  databaseStore.ts
  workbenchStore.ts

composables/
  useAuth.ts
  useDatabaseTree.ts
  useDataGrid.ts
  useQueryBuilder.ts
  useDashboardLayout.ts

layouts/
  WorkbenchLayout.vue

views/
  Login.vue
  Register.vue
  Home.vue
  DatabaseWorkbench.vue
  DatabaseDashboard.vue
  TableDesigner.vue
  TableData.vue
  QueryBuilder.vue

components/
  ParticleBackground.vue
  AppHeader.vue
  UserAvatar.vue
  DbTree.vue
  DataGrid.vue
  DataGridToolbar.vue
  ColumnDesigner.vue
  QueryBuilderPanel.vue
  DashboardWidget.vue
  ChartCard.vue
  ConfirmDialog.vue
```

如果暂时不引入 Pinia，可以先用 composable + local state；当页面状态变复杂时再引入 Pinia。正式设计推荐使用 Pinia 管理登录、偏好、数据库树和工作台布局。

### 1.14.2 14.2 authStore

状态：

```
type AuthState = {
  accessToken: string | null;
  user: {
    userID: number;
    accountNo: string;
    userName: string;
    avatarUrl?: string;
  } | null;
};
```

行为：

- login。
- refreshAccessToken。
- logout。
- fetchMe。

### 1.14.3 14.3 request.ts

职责：

- 自动注入 Access Token。
- 401 时尝试调用 /auth/refresh。
- refresh 成功后重放原请求。
- refresh 失败后清理登录状态并跳转登录页。
- 展示后端精准错误 message。

需要避免刷新风暴：

多个请求同时 401 时，只允许一个 refresh 请求进行；其他请求等待 refresh 完成后重试。

### 1.14.4 14.4 WorkbenchLayout

布局：

顶部 AppHeader

左侧 DbTree

中间 RouterView / 动态工作区

右侧可选信息栏

背景 ParticleBackground

职责：

- 加载用户偏好。
- 应用主题变量。
- 加载数据库树。
- 提供全局刷新入口。
- 提供退出登录入口。

#### 1.14.5 14.5 DbTree

节点类型：

```
type DbTreeNode =
  | { type: 'database'; databaseId: number; displayName: string }
  | { type: 'table'; databaseId: number; tableName: string }
  | { type: 'view'; databaseId: number; viewName: string };
```

交互：

- 点击数据库打开数据库看板。
- 点击表打开表数据页。
- 右键数据库：创建表、刷新、删除数据库。
- 右键表：查看结构、打开数据、删除表。

#### 1.14.6 14.6 DataGrid

职责：

- 显示 API-ROW-001 返回的 columns 和 items。
- 显示预览接口返回的 columns、rows、total 和 hasMore。
- 支持分页预览和字段筛选。
- 支持行内编辑草稿态。
- 支持单行更新、取消。
- 支持新增行。
- 支持基于主键删除单行。
- 支持排序、列显示控制、批量新增、基于主键列表的批量修改和批量删除。

行状态：

```
type DataGridRowState = {
  rowKey: string;
  original: Record<string, unknown>;
```

```

draft: Record<string, unknown>;
changedFields: Record<string, unknown>;
status: 'clean' | 'dirty' | 'saving' | 'error';
};

```

主键生成 rowKey:

单主键: String(row[pk])

复合主键: JSON.stringify(pkValues)

无主键: 只读模式

### 1.14.7 14.7 QueryBuilderPanel

职责:

- 通过 GUI 构建查询 AST。
- 支持主表、JOIN、字段、条件、分组、聚合、排序、分页。
- 支持子查询节点编辑。
- 在前端做基础结构校验。
- 提交给后端做最终校验和执行。

前端只负责构建 AST, 不负责拼 SQL。

### 1.14.8 14.8 DashboardWidget

工作台卡片类型:

```

stat-number
line-chart
bar-chart
pie-chart
recent-operations
database-shortcut
quick-action

```

布局状态:

```

type DashboardWidgetState = {
  id: string;
  widgetType: string;
  title: string;
  x: number;
  y: number;
  width: number;
  height: number;
  config: Record<string, unknown>;
}

```

```
    visible: boolean;  
};
```

第一阶段可以先支持显示、隐藏、排序；拖拽布局作为后续增强。

## 1.15 15. 前后端交互流程

### 1.15.1 15.1 登录后进入工作台

Login.vue

```
-> authApi.login  
-> 保存 accessToken  
-> router.push('/home')
```

Home.vue / WorkbenchLayout

```
-> authApi.me  
-> preferenceApi.get  
-> workbenchApi.overview  
-> databaseApi.list  
-> 渲染工作台
```

### 1.15.2 15.2 创建数据库

用户点击创建数据库

```
-> 前端校验名称格式  
-> POST /databases  
-> 后端校验 token  
-> 后端检查容量限制  
-> 后端 CREATE DATABASE  
-> 后端写 user_databases  
-> 后端写 audit_logs  
-> 前端刷新 DbTree
```

### 1.15.3 15.3 行内编辑提交

用户编辑单元格

```
-> DataGrid 标记 dirty  
-> 用户点击行级更新  
-> PATCH /rows { primaryKey, set }  
-> 后端校验归属、表、字段、主键  
-> 后端 UPDATE
```

- > 后端写 `audit_logs`
- > 前端更新 `original`

#### 1.15.4 15.4 结构化查询

用户在 `QueryBuilder` 中配置查询

- > 前端生成 `AST`
- > 前端基础校验
- > `POST /query/select`
- > 后端 `AST` 校验
- > 后端作用域解析
- > 后端生成 `SELECT + params`
- > 后端执行
- > 返回 `columns、items、pagination`

### 1.16 16. 开发落地顺序

#### 1.16.1 Phase 1: 认证与工作台地基

- `env.ts` 完整化。
- `traceMiddleware`。
- `errorMiddleware`。
- 双 Token 设计中的 Access Token 先落地, Refresh Token 可紧随其后。
- `GET /auth/me`。
- `Home.vue / WorkbenchLayout` 骨架。

#### 1.16.2 Phase 2: 用户偏好和资源

- `user_profiles`。
- `user_preferences`。
- 头像和背景资源元信息。
- 主题色和背景配置。

#### 1.16.3 Phase 3: 数据库管理

- `user_databases`。
- 数据库列表。
- 创建数据库。
- 删除数据库。
- 容量限制。
- 审计日志。



#### 1.16.4 Phase 4: 表结构管理

- 表列表。
- 创建表。
- 表结构查看。
- 删除表。
- 字段、索引和约束管理。
- 视图逐步扩展。

当前实现范围：

- 已完成数据库对象列表、结构化创建表、表结构读取、删除表和在线结构管理。
- 创建表阶段支持字段、可选主键、唯一约束、外键、有限取值检查约束和普通/唯一索引的服务端校验与 DDL 生成。
- 在线结构管理支持字段新增、字段修改、字段删除、索引新增/删除、约束新增/删除，其中有限取值通过 CHECK 约束表达。
- 视图管理仍随 GUI 查询构造器后续落地。

#### 1.16.5 Phase 5: 表数据 CRUD

- 分页预览和字段筛选。
- 排序和列显示控制。
- 单行新增。
- 批量新增。
- 行内编辑草稿态。
- 基于主键的行级更新。
- 基于主键列表的批量修改。
- 基于主键的单行删除。
- 基于主键列表的批量删除。
- 无主键表浏览、新增和只读保护。
- 基于主键列表的批量写入限制、二次确认和事务原子性。

#### 1.16.6 Phase 6: GUI 查询构造器

- 基础 SELECT：已落地。
- JOIN：已落地普通表单和后端校验。
- 聚合：已落地 COUNT、SUM、AVG、MIN、MAX 白名单。
- 分组：已落地。
- HAVING：已落地普通表单和后端校验。
- 子查询：已落地后端 AST，前端通过高级 AST 面板表达派生表、IN、EXISTS 和标量子查询。
- 查询复杂度限制：已落地 AST 体积、JOIN 数、子查询深度、分页大小和执行时间提示。

### 1.16.7 Phase 7: 统计和 DIY 工作台

V7 首轮目标是“可用的统计 Dashboard”，而不是一次性做成无限自定义门户。实现时必须保证现有 Home、数据库工作台、表数据预览和 GUI 查询能力不回退。

小阶段：

| 小阶段  | 后端交付                                                                | 前端交付                   | 验收重点                 |
|------|---------------------------------------------------------------------|------------------------|----------------------|
| V7.1 | <code>statsService</code> 、 <code>auditService.list</code> 、统计与审计路由 | 统计接口工具函数、审计列表数据模型      | 接口鉴权、归属校验、分页、容量限制    |
| V7.2 | 用户级统计聚合                                                             | 首页统计卡片、操作趋势、数据库分布、最近操作 | 首页只放总览、入口和统计，不放表内部操作 |
| V7.3 | 数据库级和表级统计聚合                                                         | 工作台统计入口、表结构窗口的统计区域     | 大表不阻塞，空表和无权限错误可读     |
| V7.4 | 偏好接口支持<br><code>dashboardLayout</code>                              | 卡片显示、隐藏、排序和保存          | 刷新后布局保持，主题色全局生效      |
| V7.5 | 统一错误、审计、性能收口                                                        | 响应式、加载态、空状态、错误态统一      | 构建通过，横向滚动盡量避免        |

首轮纳入：

- 用户级统计：数据库数量、表数量、视图数量、索引数量、估算行数、容量占用、操作趋势、最近操作。
- 数据库级统计：表/视图/索引数量、数据大小、索引大小、表大小 Top N、字段类型分布。
- 表级统计：字段数量、主键、外键、索引、估算行数、空值比例、数值字段聚合、分类字段 Top N。
- 审计日志列表：分页、时间范围、对象类型、操作类型、数据库过滤、`traceId` 检索。
- Dashboard DIY：卡片显示、隐藏、排序和卡片级配置持久化。

首轮不纳入：

- 自由拖拽网格。
- 多套布局方案切换。
- 用户自定义脚本卡片。
- 后台定时统计任务。
- 对超大表执行无上限 `COUNT(*)` 或全表字段扫描。

统计实现原则：

- 用户级和数据库级优先使用 `information_schema` 估算值。
- 表级聚合必须限制字段数量、Top N 数量、采样行数或执行时间。
- 所有字段名、表名、排序字段必须来自后端白名单，不接受前端传 SQL 片段。
- 统计访问写入审计日志，但审计详情不保存完整查询参数中的敏感值。

### 1.17 17. 实现检查清单

后端实现一个接口时必须检查：

- 是否经过 `authMiddleware`。
- 是否生成 `traceId`。
- 是否校验数据库归属。
- 是否校验标识符。
- 是否使用 `SQL builder`。
- 是否参数化字段值。
- 是否执行容量限制。
- 是否记录审计日志。
- 是否返回统一响应。
- 是否映射 `MySQL` 错误。

前端实现一个页面时必须检查：

- 是否使用 `request` 统一请求。
- 是否处理 `loading`。
- 是否展示后端精准错误。
- 是否处理 `401` 自动刷新。
- 是否避免直接拼 `SQL`。
- 是否把待提交修改和已保存数据区分。
- 是否给高风险操作二次确认。

### 1.18 18. 非功能性设计

#### 1.18.1 18.1 性能设计

- 所有列表接口必须分页，默认 `pageSize = 20`，最大 `pageSize = 100`。
- 查询构造器最大返回 1000 行，超过必须分页或缩小条件。
- 查询最大执行时间默认 10 秒，超时后返回容量或超时错误。
- 统计接口优先使用 `information_schema` 的估算数据，避免对大表频繁执行 `COUNT(*)`。
- 表格数据页只加载当前页，不一次性拉取整表。

#### 1.18.2 18.2 可用性设计

- `MySQL` 不可用时，数据库管理能力降级为明确错误提示，前端不能白屏。
- `Access Token` 过期时，前端先尝试刷新 `Token`；刷新失败后才跳转登录页。
- 高风险操作必须给出影响说明和确认方式。
- 表格行内编辑必须支持取消，避免误提交。

### 1.18.3 18.3 可维护性设计

- controller 只处理请求和响应，不拼接 SQL。
- SQL 构造集中在 sql/ 目录。
- 错误响应集中在 errorMiddleware。
- 容量限制集中在 limits.ts 和 limitService。
- 前端 API 调用集中在 api/ 目录。

### 1.18.4 18.4 可观测性设计

- 每个请求都有 traceId。
- 关键操作写入 audit\_logs。
- 后端日志、接口响应、审计日志都包含同一个 traceId。
- MySQL 错误必须映射为可读错误码。

### 1.18.5 18.5 可扩展性设计

- 用户物理库通过 user\_databases 映射，不把真实库名暴露给前端。
- 工作台卡片使用 widgetType + config 模型，后续可以扩展新卡片。
- 查询构造器使用 AST，后续可以扩展更多查询节点。

## 1.19 19. 防御性设计

### 1.19.1 19.1 输入防御

- 所有请求参数必须经过 controller 或 validator 校验。
- 数据库名、表名、字段名、索引名、约束名、视图名必须符合标识符规则。
- 文件上传必须校验 MIME、扩展名、大小和文件头。

### 1.19.2 19.2 SQL 防御

- 字段值只能通过参数化传入。
- 标识符只能来自校验后的用户输入或 MySQL 元数据。
- 业务代码不能手写反引号拼接 SQL。
- 查询构造器只生成 SELECT，不生成写操作 SQL。

### 1.19.3 19.3 权限防御

- 所有数据库相关接口都先执行 databaseResolver。
- 事务、资源、偏好、审计查询都必须校验归属。
- 不向前端返回真实物理库名、文件绝对路径、Token 哈希。

#### 1.19.4 19.4 状态防御

- Refresh Token 使用轮换机制。
- 事务必须有超时时间，超时自动回滚。
- 创建数据库时如果系统表写入失败，必须尝试补偿删除物理库。
- 行内编辑必须先进入草稿态，只有显式提交才写数据库。

#### 1.19.5 19.5 容量防御

- 创建类操作先检查容量，再执行写入。
- 查询类操作检查 AST 大小、JOIN 数、子查询深度、分页大小和执行时间。
- 批量修改和删除先预估影响行数，超过阈值要求确认或拒绝。

### 1.20 20. 详细设计追踪

为避免追踪信息在 PDF 宽表中挤压，详细设计追踪按设计项列出：

- 双 Token：对应需求 REQ-AUTH；对应接口 API-AUTH；对应安全 SEC-AUTH。
- 工作台偏好：对应需求 REQ-WORKBENCH、REQ-PREF；对应接口 API-WORKBENCH、API-PREF；对应安全 SEC-AUTHZ。
- 用户资源：对应需求 REQ-ASSET；对应接口 API-ASSET；对应安全 SEC-FILE。
- 数据库管理：对应需求 REQ-DB；对应接口 API-DB；对应安全 SEC-AUTHZ、SEC-SQL。
- 表结构管理：对应需求 REQ-TABLE；对应接口 API-TABLE；对应安全 SEC-SQL、SEC-CONFIRM。
- 行内编辑：对应需求 REQ-ROW；对应接口 API-ROW；对应安全 SEC-SQL、SEC-CONFIRM。
- 查询构造器：对应需求 REQ-QUERY；对应接口 API-QUERY；对应安全 SEC-SQL、SEC-LIMIT。
- 事务：对应需求 REQ-TX；对应接口 API-TX；对应安全 SEC-TX。
- 统计：对应需求 REQ-STATS；对应接口 API-STATS；对应安全 SEC-AUTHZ、SEC-LIMIT。
- 审计：对应需求 REQ-AUDIT；对应接口 API-AUDIT；对应安全 SEC-AUDIT。
- 容量限制：对应需求 REQ-LIMIT；对应接口规范 2.4；对应安全 SEC-LIMIT。

## 2 数据库设计附录

### 3 数据库设计

#### 3.1 1. 设计原则

##### 3.1.1 DATA-001 系统库和用户物理库分离

系统库 DB\_App 保存平台元数据。用户物理库保存用户自行创建的真实数据。

### 3.1.2 DATA-002 第三范式

系统库核心表满足第三范式：

- `users` 只保存认证信息。
- `user_profiles` 保存展示资料。
- `user_preferences` 保存偏好设置。
- `user_assets` 保存资源元信息。
- `user_databases` 保存数据库归属关系。
- `refresh_tokens` 保存会话刷新信息。
- `audit_logs` 保存操作追踪。

### 3.1.3 DATA-003 用户表结构自由

用户物理库中的表由用户通过 GUI 自行设计。系统只限制安全、容量和权限，不限制用户表的业务字段。

### 3.1.4 DATA-004 真实物理库名后端生成

前端只使用平台数据库 ID。真实 MySQL database 名只由后端生成和使用。

## 3.2 2. 系统库表总览

| 表名                                | 说明              |
|-----------------------------------|-----------------|
| <code>users</code>                | 用户认证表           |
| <code>user_profiles</code>        | 用户资料表           |
| <code>user_preferences</code>     | 用户偏好表           |
| <code>user_assets</code>          | 用户资源元信息表        |
| <code>refresh_tokens</code>       | Refresh Token 表 |
| <code>user_databases</code>       | 用户数据库归属表        |
| <code>transaction_sessions</code> | 事务会话表           |
| <code>audit_logs</code>           | 操作审计日志表         |
| <code>stat_snapshots</code>       | 统计快照表           |

## 3.3 3. 命名规则

数据库显示名、表名、字段名、索引名、约束名、视图名统一使用：

`^[A-Za-z][A-Za-z0-9_]{0,63}$`

规则：

- 必须以英文字母开头。
- 只允许英文字母、数字、下划线。
- 最长 64 字符。
- 不允许空格、中文、横线、点号、反引号。

### 3.4 4. users 用户认证表

```
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY COMMENT '用户内部唯一 ID',
  user_name VARCHAR(50) NOT NULL COMMENT '用户名, 仅供展示',
  account_no VARCHAR(10) NOT NULL COMMENT '登录账号, 10 位随机数字',
  password_hash VARCHAR(255) NOT NULL COMMENT 'bcrypt 密码哈希值',
  status ENUM('active', 'disabled') NOT NULL DEFAULT 'active' COMMENT '用户状态',
  register_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '账号注册时间',
  last_login_at TIMESTAMP NULL DEFAULT NULL COMMENT '最近登录时间',

  UNIQUE KEY uk_users_account_no (account_no),
  KEY idx_users_status (status)
) COMMENT='系统用户表';
```

### 3.5 5. user\_profiles 用户资料表

```
CREATE TABLE IF NOT EXISTS user_profiles (
  user_id INT PRIMARY KEY COMMENT '用户 ID',
  nickname VARCHAR(50) NOT NULL COMMENT '展示昵称',
  avatar_asset_id BIGINT NULL COMMENT '头像资源 ID',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',

  CONSTRAINT fk_profiles_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE
) COMMENT='用户资料表';
```

### 3.6 6. user\_preferences 用户偏好表

```
CREATE TABLE IF NOT EXISTS user_preferences (
  user_id INT PRIMARY KEY COMMENT '用户 ID',
  theme_color VARCHAR(20) NOT NULL DEFAULT '#409EFF' COMMENT '主题色',
  background_mode ENUM('particle', 'image', 'gradient') NOT NULL DEFAULT 'particle' COMMENT '背景模式',
  background_asset_id BIGINT NULL COMMENT '背景资源 ID',
```

```

layout_json JSON NULL COMMENT '工作台布局配置, 保存主题模式和分页大小等可扩展偏好',
confirm_preferences_json JSON NULL COMMENT '二次确认偏好',
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '

CONSTRAINT fk_preferences_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE
) COMMENT='用户偏好表';

```

说明:

- 删除数据库、删除表、删除字段属于强制确认, 不由偏好关闭。
- 批量插入、批量修改、批量删除、级联删除提示由偏好控制。
- V7 起 layout\_json 增加 dashboardLayout, 用于保存 Dashboard 卡片显示、隐藏、排序和卡片级配置。

### 3.7 7. user\_assets 用户资源元信息表

```

CREATE TABLE IF NOT EXISTS user_assets (
    id BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '资源 ID',
    user_id INT NOT NULL COMMENT '用户 ID',
    asset_type ENUM('avatar', 'background', 'other') NOT NULL COMMENT '资源类型',
    storage_type VARCHAR(30) NOT NULL DEFAULT 'local' COMMENT '存储类型',
    storage_key VARCHAR(500) NOT NULL COMMENT '存储键',
    original_name VARCHAR(255) NULL COMMENT '原始文件名',
    mime_type VARCHAR(100) NOT NULL COMMENT 'MIME 类型',
    file_size BIGINT UNSIGNED NOT NULL COMMENT '文件大小, 字节',
    checksum VARCHAR(128) NOT NULL COMMENT '文件校验和',
    status ENUM('active', 'deleted') NOT NULL DEFAULT 'active' COMMENT '状态',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
    deleted_at DATETIME NULL COMMENT '删除时间',

    CONSTRAINT fk_assets_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE,
    KEY idx_assets_user_type (user_id, asset_type),
    KEY idx_assets_user_time (user_id, created_at),
    KEY idx_assets_status (status)
) COMMENT='用户资源元信息表';

```

说明:

- 当前存储类型固定写入 local。



- storage\_key 保存服务端生成的资源键。
- 数据库不保存图片二进制。

### 3.8 8. refresh\_tokens 会话刷新表

```
CREATE TABLE IF NOT EXISTS refresh_tokens (
  id BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT 'Refresh Token ID',
  user_id INT NOT NULL COMMENT '用户 ID',
  token_hash CHAR(64) NOT NULL COMMENT 'Refresh Token SHA-256 哈希',
  token_family CHAR(36) NOT NULL COMMENT 'Token 家族 ID',
  expires_at TIMESTAMP NOT NULL COMMENT '过期时间',
  revoked_at TIMESTAMP NULL DEFAULT NULL COMMENT '撤销时间',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',

  CONSTRAINT fk_refresh_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE,
  UNIQUE KEY idx_refresh_token_hash (token_hash),
  KEY idx_refresh_user_family (user_id, token_family),
  KEY idx_refresh_user_expires (user_id, expires_at)
) COMMENT='Refresh Token 会话表';
```

### 3.9 9. user\_databases 用户数据库归属表

```
CREATE TABLE IF NOT EXISTS user_databases (
  id BIGINT AUTO_INCREMENT PRIMARY KEY COMMENT '用户数据库元数据 ID',
  user_id INT NOT NULL COMMENT '用户 ID',
  display_name VARCHAR(64) NOT NULL COMMENT '用户可见数据库名',
  schema_name VARCHAR(128) NOT NULL COMMENT '实际 MySQL schema 名称',
  status ENUM('active', 'deleting', 'deleted') NOT NULL DEFAULT 'active' COMMENT '数据库状态',
  table_count INT NOT NULL DEFAULT 0 COMMENT '表数量快照',
  size_bytes BIGINT NOT NULL DEFAULT 0 COMMENT '容量快照',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
  deleted_at TIMESTAMP NULL DEFAULT NULL COMMENT '删除时间',

  CONSTRAINT fk_user_databases_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE,
  UNIQUE KEY uk_user_databases_schema_name (schema_name),
  UNIQUE KEY uk_user_databases_user_display_status (user_id, display_name, status),
```

```
KEY idx_user_databases_user_status_time (user_id, status, created_at)
) COMMENT='用户数据库元数据表';
```

物理库命名格式:

```
u{userId}_{safeDisplayName}_{randomSuffix}
```

说明:

- `schema_name` 只允许后端内部使用, 前端和接口响应不返回真实物理库名。
- `status='deleting'` 表示正在执行物理库删除, 用于防止删除失败时状态与 MySQL schema 不一致。
- `table_count` 和 `size_bytes` 是快照字段; 列表接口优先从 `information_schema.tables` 读取实时值。
- 服务启动时会检查并创建 `user_databases`, 避免已有开发库漏执行新版 `init.sql`。

### 3.10 10. transaction\_sessions 事务会话表

```
CREATE TABLE IF NOT EXISTS transaction_sessions (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY COMMENT '事务会话 ID',
  transaction_key VARCHAR(64) NOT NULL UNIQUE COMMENT '对外事务 ID',
  user_id INT NOT NULL COMMENT '用户 ID',
  database_id BIGINT NOT NULL COMMENT '平台数据库 ID',
  isolation_level ENUM('READ COMMITTED', 'REPEATABLE READ', 'SERIALIZABLE') NOT NULL DEFAULT 'R',
  status ENUM('active', 'committed', 'rolled_back', 'expired') NOT NULL DEFAULT 'active' COMMENT '状态',
  started_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '开始时间',
  expires_at DATETIME NOT NULL COMMENT '过期时间',
  finished_at DATETIME NULL COMMENT '结束时间',

  INDEX idx_transaction_user_status (user_id, status),
  INDEX idx_transaction_database_status (database_id, status),
  INDEX idx_transaction_expires (expires_at),

  CONSTRAINT fk_transaction_user
    FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_transaction_database
    FOREIGN KEY (database_id) REFERENCES user_databases(id)
) COMMENT='事务会话表';
```

说明:

- `transaction_key` 是前端使用的事务 ID。
- 实际数据库连接由后端内存连接池管理。
- 表中记录用于追踪事务生命周期。

### 3.11 11. audit\_logs 操作日志表

```
CREATE TABLE IF NOT EXISTS audit_logs (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY COMMENT '日志 ID',
  trace_id VARCHAR(64) NOT NULL COMMENT '请求追踪 ID',
  user_id INT NULL COMMENT '操作用户 ID',
  database_id BIGINT NULL COMMENT '平台数据库 ID',
  transaction_id BIGINT UNSIGNED NULL COMMENT '事务会话 ID',
  object_type VARCHAR(30) NOT NULL COMMENT '对象类型',
  object_name VARCHAR(128) NULL COMMENT '对象名称',
  action_type VARCHAR(50) NOT NULL COMMENT '操作类型',
  summary VARCHAR(255) NOT NULL COMMENT '操作摘要',
  detail_json JSON NULL COMMENT '操作详情',
  success BOOLEAN NOT NULL DEFAULT TRUE COMMENT '成功状态',
  error_code VARCHAR(80) NULL COMMENT '错误码',
  error_message VARCHAR(500) NULL COMMENT '错误消息',
  duration_ms INT UNSIGNED NULL COMMENT '耗时毫秒',
  ip_address VARCHAR(45) NULL COMMENT '客户端 IP',
  user_agent VARCHAR(255) NULL COMMENT 'User-Agent',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',

  INDEX idx_audit_trace (trace_id),
  INDEX idx_audit_user_time (user_id, created_at),
  INDEX idx_audit_database_time (database_id, created_at),
  INDEX idx_audit_action_type (action_type),
  INDEX idx_audit_object_type (object_type),

  CONSTRAINT fk_audit_user
    FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_audit_database
    FOREIGN KEY (database_id) REFERENCES user_databases(id),
  CONSTRAINT fk_audit_transaction
    FOREIGN KEY (transaction_id) REFERENCES transaction_sessions(id)
) COMMENT='操作审计日志表';
```

object\_type 值:

AUTH

USER

PREFERENCE

DATABASE

TABLE

COLUMN

CONSTRAINT

INDEX

VIEW

ROW

QUERY

TRANSACTION

STATS

ASSET

action\_type 值:

AUTH\_REGISTER

AUTH\_LOGIN

AUTH\_REFRESH

AUTH\_LOGOUT

PREFERENCE\_UPDATE

DB\_CREATE

DB\_UPDATE

DB\_DELETE

TABLE\_CREATE

TABLE\_ALTER

TABLE\_DELETE

COLUMN\_CREATE

COLUMN\_UPDATE

COLUMN\_DELETE

CONSTRAINT\_CREATE

CONSTRAINT\_DELETE

INDEX\_CREATE

INDEX\_DELETE

VIEW\_CREATE

VIEW\_UPDATE

VIEW\_DELETE

ROW\_SELECT

ROW\_INSERT

ROW\_UPDATE

ROW\_DELETE

QUERY\_SELECT

TX\_BEGIN

TX\_COMMIT

TX\_ROLLBACK

STATS\_VIEW

ASSET\_VIEW

### 3.12 12. stat\_snapshots 统计快照表

```
CREATE TABLE IF NOT EXISTS stat_snapshots (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY COMMENT '快照 ID',
  user_id INT NOT NULL COMMENT '用户 ID',
  database_id BIGINT NULL COMMENT '平台数据库 ID',
  table_name VARCHAR(64) NULL COMMENT '表名',
  scope ENUM('user', 'database', 'table') NOT NULL COMMENT '统计范围',
  metric_json JSON NOT NULL COMMENT '统计指标',
  captured_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '采集时间',

  INDEX idx_stat_user_scope_time (user_id, scope, captured_at),
  INDEX idx_stat_database_time (database_id, captured_at),

  CONSTRAINT fk_stat_user
    FOREIGN KEY (user_id) REFERENCES users(id),
  CONSTRAINT fk_stat_database
    FOREIGN KEY (database_id) REFERENCES user_databases(id)
) COMMENT='统计快照表';
```

### 3.13 13. MySQL 元数据来源

表、字段、索引、约束、视图信息以 MySQL 元数据为准。

表与视图：

```
SELECT table_name, table_type, table_rows, data_length, index_length, create_time, update_time
FROM information_schema.tables
WHERE table_schema = ?;
```

字段：

```
SELECT
  column_name,
  data_type,
  column_type,
  is_nullable,
  column_key,
  column_default,
  extra,
  ordinal_position
FROM information_schema.columns
WHERE table_schema = ?
  AND table_name = ?
```

```
ORDER BY ordinal_position;
```

索引:

```
SELECT
    index_name,
    non_unique,
    seq_in_index,
    column_name,
    collation,
    index_type
FROM information_schema.statistics
WHERE table_schema = ?
    AND table_name = ?
ORDER BY index_name, seq_in_index;
```

约束:

```
SELECT
    constraint_name,
    constraint_type
FROM information_schema.table_constraints
WHERE table_schema = ?
    AND table_name = ?;
```

外键:

```
SELECT
    constraint_name,
    column_name,
    referenced_table_name,
    referenced_column_name
FROM information_schema.key_column_usage
WHERE table_schema = ?
    AND table_name = ?
    AND referenced_table_name IS NOT NULL;
```

视图:

```
SELECT table_name, view_definition, check_option, is_updatable
FROM information_schema.views
WHERE table_schema = ?
    AND table_name = ?;
```

### 3.14 14. 支持的数据类型

GUI 字段类型清单：

TINYINT  
SMALLINT  
INT  
BIGINT  
DECIMAL  
FLOAT  
DOUBLE  
CHAR  
VARCHAR  
TEXT  
MEDIUMTEXT  
LONGTEXT  
DATE  
TIME  
DATETIME  
TIMESTAMP  
BOOLEAN  
ENUM  
JSON  
BLOB  
MEDIUMBLOB  
LONGBLOB

字段属性：

- unsigned
- length
- precision
- scale
- nullable
- defaultValue
- autoIncrement
- comment
- enumValues: 枚举式有限取值，落地时优先转换为 MySQL ENUM 或受控 CHECK 约束。
- checkExpression: 检查约束表达式，必须由 GUI 结构化配置并由后端白名单生成 SQL。

### 3.15 15. 索引设计

#### 3.15.1 DATA-IDX-001 索引设计原则

系统库索引必须服务于高频查询、归属校验、唯一性约束和审计检索。

索引设计原则：

- 主键使用自增 ID 或稳定唯一 ID。
- 外键字段必须建立索引。
- 高频登录字段必须唯一索引。
- 高频按用户查询的表必须包含 `user_id` 前缀索引。
- 高频时间列表查询使用组合索引，例如 `(user_id, created_at)`。
- 不为低选择性布尔字段单独建索引，除非与高选择性字段组合。
- JSON 字段默认不建索引；若后续需要查询 JSON 内部字段，再考虑生成列索引。

#### 3.15.2 DATA-IDX-002 系统库索引清单

为避免长索引名在 PDF 表格中挤压列宽，本清单按表分组维护；每条记录仍保持“索引、类型、目的”三项信息。

- `users`
  - `uk_users_account_no(account_no)`: 唯一索引；登录账号唯一和登录查询。
  - `idx_users_status(status)`: 普通索引；后台筛选用户状态，当前阶段可选。
- `user_profiles`
  - `pk_user_profiles(user_id)`: 主键/唯一；一个用户一份资料。
- `user_preferences`
  - `pk_user_preferences(user_id)`: 主键/唯一；一个用户一份偏好。
- `user_assets`
  - `idx_assets_user_type(user_id, asset_type)`: 普通索引；查询用户头像、背景等资源。
  - `idx_assets_user_time(user_id, created_at)`: 普通索引；用户资源列表。
- `refresh_tokens`
  - `idx_refresh_token_hash(token_hash)`: 唯一索引；快速校验 Refresh Token。
  - `idx_refresh_user_family(user_id, token_family)`: 普通索引；Token 家族撤销。
  - `idx_refresh_user_expires(user_id, expires_at)`: 普通索引；清理过期 Token。
- `user_databases`
  - `uk_user_databases_schema_name(schema_name)`: 唯一索引；真实物理库名唯一。
  - `uk_user_databases_user_display_status(user_id, display_name, status)`: 唯一索引；同一用户同一状态下数据库显示名唯一。
  - `idx_user_databases_user_status_time(user_id, status, created_at)`: 普通索引；查询当前用户有效数据库并按创建时间排序。
- `transaction_sessions`
  - `idx_tx_user_status(user_id, status)`: 普通索引；查询用户活跃事务。



- idx\_tx\_expires\_at(expires\_at): 普通索引; 超时事务扫描。
- audit\_logs
  - idx\_audit\_user\_time(user\_id, created\_at): 普通索引; 用户操作日志分页。
  - idx\_audit\_database\_time(database\_id, created\_at): 普通索引; 数据库级操作日志。
  - idx\_audit\_action\_time(action\_type, created\_at): 普通索引; 操作类型统计。
  - idx\_audit\_trace\_id(trace\_id): 普通索引; 根据 traceId 排查问题。
- stat\_snapshots
  - idx\_stat\_user\_scope\_time(user\_id, scope, captured\_at): 普通索引; 用户统计趋势。
  - idx\_stat\_database\_time(database\_id, captured\_at): 普通索引; 数据库统计趋势。

### 3.15.3 DATA-IDX-003 用户物理库索引策略

用户创建表时, 索引由 GUI 显式配置。系统不自动为所有字段创建索引。

默认策略:

- 主键必须创建主键索引。
- 唯一约束必须创建唯一索引。
- 外键字段建议创建普通索引; 若 MySQL 自动创建则复用。
- 用户显式创建普通索引或唯一索引时, 字段必须存在。
- 单表索引数量默认最多 16 个。
- 复合索引字段顺序由用户在 GUI 中配置。

### 3.15.4 DATA-IDX-004 索引变更安全

创建或删除索引前必须:

- 校验数据库归属。
- 校验表存在。
- 校验字段存在。
- 校验索引名符合标识符规则。
- 校验单表索引数量上限。
- 对删除唯一索引、删除外键相关索引等高风险操作要求二次确认。
- 写入审计日志。

### 3.15.5 DATA-IDX-005 索引统计来源

索引元数据以 MySQL 为准:

```
SELECT
    index_name,
    non_unique,
    column_name,
```

```
seq_in_index,  
collation,  
index_type  
FROM information_schema.statistics  
WHERE table_schema = ?  
AND table_name = ?  
ORDER BY index_name, seq_in_index;
```

## 3.16 16. 容量限制

### 3.16.1 DATA-LIMIT-001 限制来源

容量限制来自服务端配置。系统库默认不保存限制值，避免把部署策略和业务数据耦合。

如果后续需要不同用户套餐，可以新增独立表：

```
plans  
user_plan_assignments
```

当前阶段不设计套餐表，统一使用服务端默认限制。

### 3.16.2 DATA-LIMIT-002 默认容量限制

| 限制项         | 默认值   | 校验依据                                                 |
|-------------|-------|------------------------------------------------------|
| 单用户数据库数量    | 10    | user_databases                                       |
| 单数据库表数量     | 100   | information_schema.tables                            |
| 单数据库视图数量    | 50    | information_schema.views                             |
| 单表字段数量      | 80    | information_schema.columns                           |
| 单表索引数量      | 16    | information_schema.statistics                        |
| 单用户物理数据总量   | 1GB   | information_schema.tables.data_length + index_length |
| 单数据库物理数据量   | 512MB | information_schema.tables                            |
| 单表物理数据量     | 256MB | information_schema.tables                            |
| 单次分页返回行数    | 100   | API 参数                                               |
| 单次查询最大返回行数  | 1000  | 查询服务层                                                |
| 单次批量插入行数    | 500   | API 请求体                                              |
| 单次批量修改影响行数  | 200   | 影响行数预估                                               |
| 单次批量删除影响行数  | 200   | 影响行数预估                                               |
| 查询最大 JOIN 数 | 8     | 查询 AST                                               |
| 查询最大子查询深度   | 3     | 查询 AST                                               |
| 查询最大执行时间    | 10 秒  | MySQL 会话级限制或应用层超时                                    |
| 查询 AST 最大体积 | 64KB  | 请求体大小                                                |
| 头像文件大小      | 2MB   | 上传服务                                                 |
| 背景图文件大小     | 8MB   | 上传服务                                                 |

| 限制项     | 默认值   | 校验依据                               |
|---------|-------|------------------------------------|
| 单用户资源总量 | 100MB | <code>user_assets.file_size</code> |

### 3.16.3 DATA-LIMIT-003 容量统计原则

- 数据库、表、视图、字段、索引数量以 MySQL 元数据为准。
- 物理容量以 `information_schema.tables` 的 `data_length + index_length` 估算。
- 用户资源容量以系统库 `user_assets.file_size` 汇总。
- 容量限制校验必须发生在写入或创建操作之前。
- 统计值允许存在短暂延迟，但限制判断必须尽量使用实时查询。

## 4 安全设计附录

## 5 安全设计

### 5.1 1. 安全目标

系统必须满足以下安全目标：

- 未登录用户不能访问工作台接口。
- 用户不能访问他人数据库。
- 前端不能直接执行任意 SQL。
- 动态 SQL 不产生注入风险。
- Access Token 泄露后的有效时间受限。
- Refresh Token 被服务端追踪和轮换。
- 高风险操作具备二次确认。
- 批量操作具备影响分析和容量限制。
- 数据库错误被精准归类并安全返回。
- 关键操作具备 `traceId` 和审计日志。

### 5.2 2. 双 Token 安全

#### 5.2.1 SEC-AUTH-001 Access Token

Access Token 用于访问普通 API。

| 属性   | 规则                                                     |
|------|--------------------------------------------------------|
| 有效期  | 15 分钟                                                  |
| 签名密钥 | <code>JWT_ACCESS_SECRET</code>                         |
| 传输方式 | <code>Authorization: Bearer &lt;accessToken&gt;</code> |

| 属性      | 规则               |
|---------|------------------|
| Payload | userID、accountNo |

Payload 禁止包含：

- 密码。
- 密码哈希。
- Refresh Token。
- 数据库列表。
- 用户资源路径。

### 5.2.2 SEC-AUTH-002 Refresh Token

Refresh Token 用于刷新 Access Token。

| 属性    | 规则                 |
|-------|--------------------|
| 有效期   | 7 天                |
| 签名密钥  | JWT_REFRESH_SECRET |
| 存放位置  | HttpOnly Cookie    |
| 服务端存储 | token 哈希           |

Cookie 属性：

```
HttpOnly = true
Secure = true
SameSite = Strict
Path = /api/v1/auth
```

开发环境通过环境变量关闭 Secure，生产环境必须开启 Secure。

### 5.2.3 SEC-AUTH-003 Refresh Token Rotation

刷新流程：

```
校验 Refresh Token
校验 token_hash 存在且未撤销
撤销旧 Refresh Token
签发新 Access Token
签发新 Refresh Token
保存新 Refresh Token 哈希
设置新 Refresh Token Cookie
写入审计日志
```

被撤销的 Refresh Token 再次出现时，系统撤销同一 token family 下的全部活跃 Refresh Token。

#### 5.2.4 SEC-AUTH-004 退出登录

退出登录流程：

校验 Refresh Token

撤销 Refresh Token

清除 Refresh Token Cookie

写入审计日志

### 5.3 3. 授权安全

#### 5.3.1 SEC-AUTHZ-001 用户上下文

所有受保护接口必须经过鉴权中间件。鉴权中间件将当前用户 ID 写入请求上下文。

#### 5.3.2 SEC-AUTHZ-002 数据库归属校验

所有数据库对象接口必须执行：

```
currentUserId + databaseId + status active
```

查询 user\_databases 成功后才能获得 schema\_name。

#### 5.3.3 SEC-AUTHZ-003 对象归属继承

表、字段、索引、约束、视图、数据行、事务均继承数据库归属权限。没有通过数据库归属校验的请求不能继续访问 MySQL 用户物理库。

#### 5.3.4 SEC-AUTHZ-004 事务归属校验

事务提交、回滚和事务内写操作必须校验：

```
transactionId + currentUserId + databaseId + status active
```

### 5.4 4. SQL 安全

#### 5.4.1 SEC-SQL-001 值参数化

所有用户输入值必须使用参数化查询。

```
WHERE `name` = ?
```

### 5.4.2 SEC-SQL-002 标识符校验

数据库显示名、表名、字段名、索引名、约束名、视图名必须符合：

`^[A-Za-z][A-Za-z0-9_]{0,63}$`

### 5.4.3 SEC-SQL-003 标识符来源校验

动态标识符必须满足：

- 数据库真实名来自 `user_databases.schema_name`。
- 表名来自 `information_schema.tables`。
- 字段名来自 `information_schema.columns`。
- 索引名来自 `information_schema.statistics`。
- 约束名来自 `information_schema.table_constraints`。
- 视图名来自 `information_schema.views`。

### 5.4.4 SEC-SQL-004 统一转义

后端通过统一工具封装反引号。业务代码不能手写反引号拼接。

### 5.4.5 SEC-SQL-005 GUI 查询限制

GUI 查询必须执行：

- 主表存在校验。
- JOIN 表存在校验。
- 派生表子查询校验。
- IN、EXISTS、标量子查询校验。
- 字段存在校验。
- 操作符白名单校验。
- 聚合函数白名单校验。
- 分组字段校验。
- 排序字段校验。
- 分页上限校验。
- JOIN 数量上限校验。
- 子查询深度上限校验。
- 查询 AST 体积上限校验。
- 查询超时控制。

GUI 查询提交的是结构化查询 AST，不直接提交原始 SQL 字符串。后端只生成 SELECT 语句，禁止在查询构造器中生成 INSERT、UPDATE、DELETE、DROP、ALTER 等写操作。

V6 已在服务端执行 AST 校验、用户数据库归属校验、字段作用域解析、标识符引用转义、参数绑定、JOIN 数量限制、子查询深度限制、分页限制和查询执行时间提示。前端高级 AST 面板也只提交结构化 JSON，不开放原始 SQL 编辑器。

#### 5.4.6 SEC-SQL-006 写操作限制

修改和删除必须携带 GUI 条件对象。后端禁止执行无条件 UPDATE 和无条件 DELETE。

批量写操作必须执行影响行数预估。超过阈值时返回 CONFIRMATION\_REQUIRED。

### 5.5 5. 高风险操作确认

#### 5.5.1 SEC-CONFIRM-001 强制确认

以下操作必须二次确认，用户偏好不能关闭：

- 删除数据库。
- 删除表。
- 删除字段。
- 删除视图。
- 触发外键级联删除。
- 事务回滚。

确认方式：

- 删除数据库输入数据库显示名。
- 删除表输入表名。
- 删除字段输入字段名。
- 删除视图输入视图名。
- 级联删除展示影响表和影响行数后确认。
- 事务回滚展示事务 ID 后确认。

#### 5.5.2 SEC-CONFIRM-002 偏好确认

以下操作达到阈值时二次确认，用户偏好控制同类提示：

- 批量插入。
- 批量修改。
- 批量删除。
- 批量结构变更。

用户在确认弹窗勾选“本类操作不再提示”后，系统更新 user\_preferences 中对应字段。

### 5.5.3 SEC-CONFIRM-003 确认失败

确认文本不匹配时返回：

```
HTTP 412
error.type = CONFIRMATION_REQUIRED
error.code = CONFIRMATION_TEXT_MISMATCH
```

## 5.6 6. 事务安全

### 5.6.1 SEC-TX-001 事务连接

事务会话绑定后端 MySQL 连接。连接只服务创建该事务的用户。

### 5.6.2 SEC-TX-002 事务超时

事务有效期固定为 120 秒。超时后系统自动回滚并释放连接。

### 5.6.3 SEC-TX-003 隔离级别

支持的隔离级别：

```
READ COMMITTED
REPEATABLE READ
SERIALIZABLE
```

### 5.6.4 SEC-TX-004 审计

事务开始、提交、回滚、超时回滚和事务内写操作都写入审计日志。

## 5.7 7. 精确错误安全

### 5.7.1 SEC-ERROR-001 错误归类

后端必须区分：

- 参数校验错误。
- 鉴权错误。
- 授权错误。
- 确认缺失。
- 容量超限。
- MySQL 连接错误。
- MySQL 约束错误。



- MySQL 执行错误。
- 服务端内部错误。

### 5.7.2 SEC-ERROR-002 MySQL 错误返回

MySQL 错误响应包含：

- `errno`
- `sqlState`
- `sqlMessage`
- `databaseName`
- `tableName`
- `columnName`
- `constraintName`

响应禁止包含：

- 数据库连接密码。
- Access Token。
- Refresh Token。
- 完整 SQL 中的敏感参数值。

### 5.7.3 SEC-ERROR-003 批量操作错误

批量操作失败时必须返回：

- 失败行下标。
- 失败字段。
- 失败原因。
- MySQL 错误号。
- SQLSTATE。
- `traceId`。

## 5.8 8. 文件资源安全

### 5.8.1 SEC-FILE-001 资源存储

资源实体保存到服务端文件目录。系统库保存资源元信息。

### 5.8.2 SEC-FILE-002 文件校验

上传资源必须校验：

- 文件大小。

- MIME 类型。
- 扩展名。
- 文件内容签名。
- 校验和。

允许的图片 MIME 类型：

```
image/png  
image/jpeg  
image/webp
```

SVG 文件固定拒绝。

### 5.8.3 SEC-FILE-003 文件名

服务端生成随机资源键。用户原始文件名只作为展示元信息保存。

### 5.8.4 SEC-FILE-004 资源归属

读取、替换、删除资源必须校验 `asset.user_id = currentUserId`。

## 5.9 9. 容量与频率限制

### 5.9.1 SEC-LIMIT-001 容量限制

服务端配置固定限制。默认值如下：

| 限制项         | 默认值   | 安全目的          |
|-------------|-------|---------------|
| 单用户数据库数量    | 10    | 防止大量创建 schema |
| 单数据库表数量     | 100   | 防止元数据膨胀       |
| 单数据库视图数量    | 50    | 防止复杂视图滥用      |
| 单表字段数量      | 80    | 防止超宽表         |
| 单表索引数量      | 16    | 防止索引滥用拖慢写入    |
| 单用户物理数据总量   | 1GB   | 控制本地 MySQL 容量 |
| 单数据库物理数据量   | 512MB | 控制单库容量        |
| 单表物理数据量     | 256MB | 控制单表容量        |
| 单次分页返回行数    | 100   | 防止接口返回过大      |
| 单次查询最大返回行数  | 1000  | 防止大结果集压垮前端    |
| 单次批量插入行数    | 500   | 控制批量写入压力      |
| 单次批量修改影响行数  | 200   | 防止误更新大量数据     |
| 单次批量删除影响行数  | 200   | 防止误删大量数据      |
| 查询最大 JOIN 数 | 8     | 控制查询复杂度       |
| 查询最大子查询深度   | 3     | 防止嵌套查询过深      |

| 限制项         | 默认值   | 安全目的      |
|-------------|-------|-----------|
| 查询最大执行时间    | 10 秒  | 防止长查询占用连接 |
| 查询 AST 最大体积 | 64KB  | 防止请求体过大   |
| 头像文件大小      | 2MB   | 控制头像上传    |
| 背景图文件大小     | 8MB   | 控制背景上传    |
| 单用户资源总量     | 100MB | 控制用户文件容量  |

超过限制时返回：

HTTP 413

```
error.type = LIMIT_EXCEEDED
```

```
error.code = LIMIT_EXCEEDED
```

错误详情必须包含超限项、当前值和最大值，不返回任何敏感路径或 SQL 文本。

### 5.9.2 SEC-LIMIT-002 频率限制

服务端限制：

- 登录尝试频率。
- Token 刷新频率。
- 创建数据库频率。
- 删除数据库频率。
- 批量写操作频率。
- GUI 查询频率。

### 5.9.3 SEC-LIMIT-003 登录失败冷却

同一账号和同一 IP 多次登录失败后，系统启用冷却。冷却期间登录接口返回 `RATE_LIMITED`。

## 5.10 10. 审计与追踪

### 5.10.1 SEC-AUDIT-001 traceId

每个请求必须生成 `traceId`。`traceId` 同时出现在：

- 接口响应。
- 服务端日志。
- 审计日志。

### 5.10.2 SEC-AUDIT-002 审计范围

审计日志记录：

- 注册。
- 登录。
- Token 刷新。
- 退出登录。
- 偏好修改。
- 数据库创建、修改、删除。
- 表创建、修改、删除。
- 字段创建、修改、删除。
- 约束创建、删除。
- 索引创建、删除。
- 视图创建、修改、删除。
- 数据查询、新增、修改、删除。
- GUI 查询。
- 事务开始、提交、回滚、超时回滚。
- 统计查看。
- 资源读取。

### 5.10.3 SEC-AUDIT-003 审计内容

审计日志包含：

- traceId。
- userId。
- databaseId。
- transactionId。
- objectType。
- objectName。
- actionType。
- summary。
- detailJson。
- success。
- errorCode。
- errorMessage。
- durationMs。
- ipAddress。
- userAgent。
- createdAt。

审计日志禁止保存密码明文、Token 明文和完整敏感参数值。

### 5.11 11. 环境变量

服务启动必须校验以下环境变量：

```
JWT_ACCESS_SECRET
JWT_REFRESH_SECRET
DB_HOST
DB_USER
DB_PASSWORD
DB_NAME
PORT
```

缺少任一变量时服务启动失败。

代码中禁止提供生产可用默认密钥。

### 5.12 12. 前端安全

前端安全要求：

- 不在前端保存 Refresh Token。
- Access Token 只用于请求头，刷新失败立即清理登录态。
- 所有危险操作必须使用统一确认组件。
- 不在 URL 中携带 Token、密码、Refresh Token 或敏感查询参数。
- 表格行内编辑只进入草稿态，点击更新按钮才提交。
- 前端错误提示优先展示后端 `message`，但不能展示敏感字段。

### 5.13 13. 防御性设计汇总

防御性设计用于处理“用户误操作、恶意输入、服务异常、数据库异常、状态不一致”等情况。

#### 5.13.1 SEC-DEF-001 默认拒绝

- 未登录请求默认拒绝。
- 无数据库归属默认拒绝。
- 不符合标识符规则的名称默认拒绝。
- 超过容量限制默认拒绝。
- 查询 AST 超过复杂度限制默认拒绝。

#### 5.13.2 SEC-DEF-002 失败安全

- 创建数据库后系统表写入失败时，必须尝试删除已创建物理库。
- 事务超时必须自动回滚。

- Refresh Token 复用疑似被盗时，撤销同一 token family。
- 批量修改和删除影响行数超过阈值时，不得直接执行。

### 5.13.3 SEC-DEF-003 最小暴露

- 不返回真实物理数据库名。
- 不返回服务端文件绝对路径。
- 不返回密码、密码哈希、Token、Token 哈希。
- MySQL 错误返回经过安全包装，只暴露可用于用户修正输入的信息。

### 5.13.4 SEC-DEF-004 可追踪

- 所有关键失败都必须带 `traceId`。
- 所有写操作成功或失败都写入审计日志。
- 审计日志不记录敏感明文。

前端必须遵守：

- 用户输入内容按文本渲染。
- 不渲染不可信 HTML。
- Access Token 优先保存在内存。
- 页面刷新后通过 Refresh Token 恢复会话。
- API 错误根据 `error.type` 和 `error.code` 精确展示。